



3
10
1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Nested Class And Final

د. غاندي هسام



برمجة ٣

RB Informatics; 23/10/2025

Nested Class

- ال Nested Class هو عبارة عن class داخلي يتم تعريفه داخل class آخر ال Class الآخر يسمى ال (Outer Class).
- ال Nested Class لا يمكن أن يُعرّف بشكل مستقل عن ال Outer Class الذي يحتويه، أي أنه يكون جزءاً من ال Outer Class ولا يمكن استخدامه بشكل منفصل أو مُستقل عنه.
- كما أن نطاق عمل ال Nested Class محصور ضمن نطاق ال Outer Class أي لا يمكن الوصول إلى ال Nested Class مباشرة خارج ال Outer Class بل يجب أن يتم الوصول إليه من خلال كائن من ال Outer Class.

أنواع ال Nested Classes :

هناك نوعان رئيسيان :

1. Non-static Nested Classes (Inner Classes):

هو ال Nested Class الذي يحتاج إلى كائن من ال Outer Class لإنشائه. أي أنه غير ثابت (non-static) ، ويعتمد على كائنات ال Outer Class للوصول إليه.

2. Static Nested Classe :

هذا النوع من ال Nested Classes يُعرّف باستخدام الكلمة المفتاحية static ، ال Static Nested Class يمكن أن تُعرّف بدون الحاجة إلى كائن من ال Outer Class فهي مترابطة مع ال Outer Class بشكل ثابت .

:Inner Class

- ال (Inner Class) يمكنها الوصول إلى جميع المتغيرات والطرق الخاصة بال (Outer Class) .

لماذا نستخدمها ؟

- لتنظيم الكود بشكل منطقي أي عندما يكون هناك علاقة قوية بين كلاسين ووجود حاجة لتوفير مجموعة من الخدمات التي تكون مطلوبة فقط من قبل ال Outer Class من الأفضل وضعها في class داخلي اخر.



مثال ب code :

```
class Outer {
    int[] nums;

    Outer(int[] n) {
        nums = n;
    }

    void analyze() {
        Inner inOb = new Inner();

        System.out.println("Minimum: " + inOb.min());
        System.out.println("Maximum: " + inOb.max());
        System.out.println("Average: " + inOb.avg());
    }
    // This is an inner class.
    class Inner {
        int min() {
            int m = nums[0];
            for (int i = 1; i < nums.length; i++) {
                if (nums[i] < m) m = nums[i];
            }
            return m;
        }

        int max() {
            int m = nums[0];
            for (int i = 1; i < nums.length; i++) {
                if (nums[i] > m) m = nums[i];
            }
            return m;
        }

        int avg() {
            int a = 0;
            for (int i = 0; i < nums.length; i++) {
                a += nums[i];
            }
            return a / nums.length;
        }
    }
}
```



```
class NestedClassDemo {
    public static void main(String[] args) {
        int[] x = { 3, 2, 1, 5, 6, 9, 7, 8 };
        Outer outOb = new Outer(x);
        outOb.analyze();
    }
}
```

Output

```
Minimum: 1
Maximum: 9
Average: 5
```

من الكود السابق نلاحظ أن ال class الداخلي استطاع الوصول واستخدام المصفوفة nums من ال class الخارجي مباشرة ، ونلاحظ أيضا ان ال class الخارجي استطاع الاستفادة والوصول الى طرق ال class الداخلي ولكن ذلك بعد انشاء كائن منه .

ملاحظة:

- ال Static Nested Class يمكنها الوصول فقط إلى الأعضاء الثابتة (static) في ال class الخارجي مباشرة. أما إذا أرادت الوصول إلى الأعضاء غير الثابتة (غير static)، فيجب عليها استخدام كائن من ال class الخارجي.
- بينما ال Non-static Nested Class أو Inner Class يمكنها الوصول إلى جميع الأعضاء (سواء كانت ثابتة أو غير ثابتة) في ال class الخارجي مباشرة، بدون الحاجة إلى كائن آخر.

Varargs Basics

- المتغيرات ذات الطول المتغير يتم تحديدها باستخدام ثلاث نقاط متتالية (...).

```
void VaTest (int... v) {
}
```

عند استخدامنا لهذا الشكل يعرف ال compiler تلقائيا أن دالة vaTest() يمكن استدعاؤها بأي عدد من الوسائط، بما في ذلك صفر من الوسائط بالإضافة إلى ذلك، يؤدي ذلك إلى أن المتغير v (وفقا للمثال السابق) يتم الإعلان عنه بشكل ضمني كمصفوفة من النوع int[] .

وبالتالي، داخل الدالة vaTest()، يتم التعامل و الوصول إلى v كما اعتدنا التعامل مع المصفوفات . يجب أن يكون المعامل (parameter) ذو الطول المتغير هو آخر معامل يتم تحديده في الدالة. يجب أن يكون هناك معامل واحد فقط من نوع Varargs في الدالة ولا يسمح تعدده .

```
class VarArgs {
    // vaTest() uses a vararg.
    static void vaTest(int ... v) {
        System.out.println("Number of args: " + v.length);
        System.out.println("Contents: ");
        for(int i=0; i < v.length; i++)
            System.out.println(" arg " + i + ": " + v[i]);

        System.out.println();
    }
    public static void main(String[] args)
    {
        // Notice how vaTest() can be called with a
        // variable number of arguments.
        vaTest(10);    // 1 arg
        vaTest(1, 2, 3); // 3 args
        vaTest();      // no args
    }
}
```

Call with different numbers of arguments.

Output

```
Number of args: 1
Contents:
  arg 0: 10
Number of args: 3
Contents:
  arg 0: 1
  arg 1: 2
  arg 2: 3
Number of args: 0
Contents:
```



Life begins after coffee



Overloading Varargs Methods:

- يمكننا استخدام مبدأ ال Overloading على دالة ما باستخدام المعاملات ذات الطول المتغير (Vararg) مثال بـ code:

```
// Varargs and overloading.
class VarArgs3 {
    static void vaTest(int... v) { → First version of vaTest()
        System.out.println("vaTest(int ...): " + "Number of args: " + v.length);
        System.out.println("Contents: ");
        for (int i = 0; i < v.length; i++) {
            System.out.println(" arg " + i + ": " + v[i]);
        }
        System.out.println();
    }
    static void vaTest(boolean... v) { → Second version of vaTest ( )
        System.out.println("vaTest(boolean ...): " + "Number of args: " + v.length);
        System.out.println("Contents: ");

        for (int i = 0; i < v.length; i++) {
            System.out.println(" arg " + i + ": " + v[i]);
        }
        System.out.println();
    }
    static void vaTest(String msg, int... v) { → Third version of vaTest()
        System.out.println("vaTest(String, int ...): " + msg + v.length);

        System.out.println("Contents: ");
        for (int i = 0; i < v.length; i++) {
            System.out.println(" arg " + i + ": " + v[i]);
        }
        System.out.println();
    }
    public static void main(String[] args) {
        vaTest(1, 2, 3);
        vaTest("Testing: ", 10, 20);
        vaTest(true, false, false);
    }
}
```

output

```

vaTest(int ...): Number of args: 3
Contents:
  arg 0: 1
  arg 1: 2
  arg 2: 3
vaTest(String, int ...): Testing: 2
Contents:
  arg 0: 10
  arg 1: 20
vaTest(boolean ...): Number of args: 3
Contents:
  arg 0: true
  arg 1: false
  arg 2: false

```



- في مثالنا هذا هناك ثلاث نسخ مختلفة من الدالة `vaTest()` باستخدام الـ `Varargs`:
- النسخة الأولى `vaTest (int... v)`: تأخذ معاملات ذات طول متغير من النوع `int`. (حيث يتم التعامل مع المتغيرات ك مصفوفة من الأعداد الصحيحة (`int[]`)).
- النسخة الثانية `vaTest (boolean... v)`: تأخذ معاملات ذات طول متغير من النوع `boolean`.
- النسخة الثالثة `vaTest (String msg, int... v)`: تأخذ أولاً معامل ثابت من النوع `String`، ثم معاملات ذات طول متغير من النوع `int`.

Varargs and Ambiguity (لبس وغموض):

- قد يحدث خطأ أثناء استدعاء دالة من النوع `Overloading Varargs Methods` وذلك بسبب حدوث غموض ولبس في تحديد الدالة المقصودة من الاستدعاء لدى الـ `compiler`.

```

class VarArgs4 {
  // Use an int vararg parameter.
  static void vaTest(int ... v) {
    // ...
  }
}

```

```
// Use a boolean vararg parameter.
static void vaTest(boolean ... v) {
    // ...
}
public static void main(String[] args)
{
    vaTest(1, 2, 3); // OK 🙌
    vaTest(true, false, false); // OK 🙌
    vaTest(); // Error: Ambiguous! ✗
}
}
```

في هذا البرنامج ال Overloading للدالة vaTest() صحيح تماما لكن مع هذا البرنامج لن يعمل بسبب حدوث خطأ أثناء الترجمة (Compile-time error).

بما أن ال vararg يمكن أن تكون فارغة، فإن هذا الاستدعاء قد يُترجم إلى استدعاء لدالة vaTest(int ...) أو إلى استدعاء لدالة vaTest(boolean ...). كلا الاستدعاءين صحيحان بنفس القدر. وبالتالي يكون الاستدعاء غامضاً بطبيعته. وكذلك في حالة وجود استدعاء الدالة vaTest() :

```
static void vaTest(int ... v) { // ...
static void vaTest(int n, int ... v) { //...
```



اللبس والغموض موجود في هذه الحالة أيضا والكومبايلر سيكون غير قادر على تحديد هل يُترجم هذا إلى استدعاء لدالة vaTest(int ...) مع معامل واحد من نوع varargs؟ أم هل يُترجم إلى استدعاء لدالة vaTest(int, int ...) بدون معاملات varargs؟

لذلك فأنه في بعض الأحيان سنحتاج إلى التخلي عن التحميل ببساطة واستخدام أسماء دوال مختلفة.

Using Final

1. Final Prevents Overriding:

الكلمة المفتاحية final يمكن استخدامها قبل الدوال لمنع قيام أي Overriding لها

```
class A {  
  
    final void meth() {  
        System.out.println("This is a final method.");  
    }  
}  
  
class B extends A {  
  
    void meth() { // ERROR! Can't override.  
        System.out.println("Illegal!");  
    }  
}
```

بما أن الدالة meth() مُعرفة ب final، فإنه لا يمكن فعل Overriding لها في الفئة B إذا حاولت القيام بذلك، سيحدث خطأ في وقت الترجمة (compile-time error).

2. Final Prevents Inheritance:

الكلمة المفتاحية FINAL تمنع الوراثة من Class وذلك بوضعها قبل تعريفه وعند تعريف كلاس بهذا الشكل فإن جميع الدوال فيه تعتبر من نوع final

ملاحظة :

من غير الصواب تعريف كلاس على أنه final و abstract في ان واحد بسبب التناقض في كل منهما حيث أن الكلاس من نوع final لا يمكن الوراثة منه ، بينما الكلاس من نوع abstract يشترط الوراثة للتنفيذ والتحقق فهو أساس لغيره من ال classes

3. Using final with Data Member:

إذا وضعت الكلمة المفتاحية final قبل اسم متغير من النوع المثل (instance variable) ، فإن قيمته لا يمكن تغييرها طوال مدة تشغيل البرنامج


```

class ErrorMsg {
    // Error codes.
    final int OUTERR = 0;
    final int INERR= 1 ; ← Declare final Constants
    final int DISKERR = 2;
    final int INDEXERR = 3;
    String[] msgs = {
        "Output Error",
        "Input Error",
        "Disk Full",
        "Index Out-Of-Bounds"
    };
    // Return the error message.
    String getErrorMsg(int i) {
        if(i >=0 & i < msgs.length)
            return msgs[i];
        else
            return "Invalid Error Code";
    }
}

class FinalD {
    public static void main(String[] args) {
        ErrorMsg err = new ErrorMsg();
        Use final constants.
        System.out.println(err.getErrorMsg(err.OUTERR));
        System.out.println(err.getErrorMsg(err.DISKERR));
    }
}

```



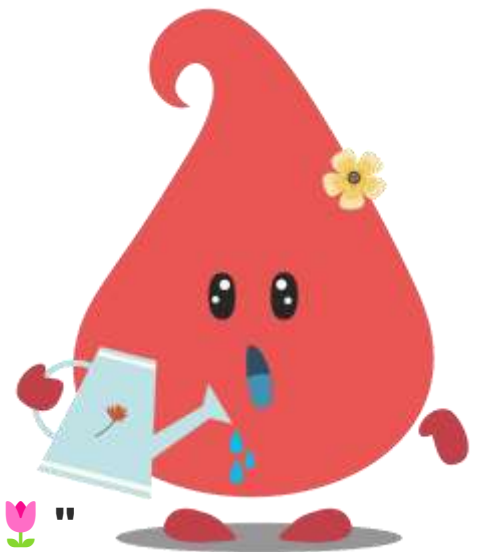
■ نلاحظ كيف يتم استخدام الثوابت final في دالة main() بما أنها أعضاء في ErrorMsg ()، و تم الوصول إليها عبر كائن من تلك الـ class، يمكن أيضاً وراثتها من قبل الـ subclass والوصول إليها مباشرة داخل تلك الـ subclass أيضاً

هل يمكن تعريف متغير على أنه **final** و **static** في ان واحد؟

٤ نعم و ذلك يسمح لنا بالاشارة اليه واستدعائه من خلال اسم الClass فقط بدلا من خلال الكائن .

هل يمكن استخدام **final** مع **Method Parameter** والمتغيرات المحلية؟

٤ نعم عندما تُستخدم الكلمة المفتاحية **final** مع **Method Parameter**، فإنها تمنع تغيير قيمتها داخل جسم الدالة وعندما تُستخدم **final** مع المتغيرات المحلية داخل الدالة أو الميثود، فإن ذلك يمنع إعادة تعيين قيمة المتغير بعد أن يتم تعيينه لأول مرة (تُستخدم هذه الميزة لضمان أن المتغير سيظل يحمل نفس القيمة طوال مدة عمله داخل الدالة)



"Don't wait for the perfect moment.

Bloom like a flower in the rain 🌸 "